

Variational Autoencoder and Conditional VAE on MNIST

Anuj Rajan Lalla (B22AI061) Abhinav Swami (B22AI003)
Swaksh Patwari (B22AI065)

Abstract

We implement a Variational Autoencoder (VAE) and a Conditional Variational Autoencoder (CVAE) in PyTorch, trained on the MNIST handwritten digit dataset. The VAE learns a smooth 20-dimensional latent space from which new digit images can be sampled, achieving a test ELBO loss of 102.79. The CVAE conditions both encoder and decoder on the digit class label (via one-hot concatenation), enabling controlled generation of specified digits with a lower test loss of 98.49. Ablations over the latent dimension $d \in \{2, 5, 10, 20, 50\}$ reveal a clear reconstruction-vs-regularization tradeoff: reconstruction BCE drops from 145.94 ($d=2$) to 77.34 ($d=50$), while KL divergence rises from 6.10 to 26.64. Latent space visualizations (t-SNE, 2D scatter, grid traversals, and interpolations) confirm that the learned representations are smooth, continuous, and semantically meaningful.

1 Introduction

A Variational Autoencoder (VAE) [1] is a generative model that learns a latent representation z of data x by jointly training an encoder $q_\phi(z|x)$ and a decoder $p_\theta(x|z)$. Unlike a standard autoencoder, the VAE imposes a probabilistic structure on the latent space by regularizing the encoder’s output toward a prior distribution $p(z) = \mathcal{N}(0, I)$. This enables meaningful sampling and interpolation in the latent space.

A key limitation of the standard VAE is that generated samples have no controllable class identity. The Conditional VAE (CVAE) [2] addresses this by conditioning both encoder and decoder on the class label y , allowing generation of images belonging to a specified class.

We implement both models and evaluate them on MNIST, studying reconstruction quality, generation capability, latent space structure, and the effect of the latent dimension.

2 Method

2.1 VAE Architecture

The encoder maps a flattened input $x \in \mathbb{R}^{784}$ through a single hidden layer ($h_e = 400$ units, ReLU activation) to produce the parameters of the approximate posterior: the mean $\mu \in \mathbb{R}^d$ and the log-variance $\log \sigma^2 \in \mathbb{R}^d$, where $d = 20$ is the latent dimension. Predicting the log-variance (rather than the raw variance) ensures numerical stability during training.

The latent vector is sampled using the **reparameterization trick**:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \tag{1}$$

where $\sigma = \exp(\frac{1}{2} \log \sigma^2)$. This makes the sampling differentiable with respect to μ and $\log \sigma^2$, enabling backpropagation through the stochastic node.

The decoder maps z through a hidden layer ($h_d = 400$, ReLU) to a sigmoid output $\hat{x} \in [0, 1]^{784}$, matching the normalized pixel range.

2.2 Loss Function

The training objective is the negative Evidence Lower Bound (ELBO):

$$\mathcal{L} = \underbrace{-\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{Reconstruction loss}} + \underbrace{D_{\text{KL}}(q_\phi(z|x) \| p(z))}_{\text{KL divergence}} \quad (2)$$

The reconstruction term is computed as the binary cross-entropy (BCE) between the input and reconstructed images, summed over pixels and averaged over the batch. For the KL divergence between the Gaussian encoder $q_\phi(z|x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ and the standard normal prior $p(z) = \mathcal{N}(0, I)$, the closed-form expression is:

$$D_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2) \quad (3)$$

2.3 CVAE Architecture

The Conditional VAE modifies both the encoder and decoder to accept the class label as an additional input:

- **Encoder input:** $[x; \text{onehot}(y)] \in \mathbb{R}^{784+10} = \mathbb{R}^{794}$
- **Decoder input:** $[z; \text{onehot}(y)] \in \mathbb{R}^{d+10} = \mathbb{R}^{30}$

where $\text{onehot}(y)$ is the 10-dimensional one-hot encoding of the digit label $y \in \{0, \dots, 9\}$. The hidden layer sizes and loss function remain identical to the VAE. The CVAE has 660,824 parameters (compared to 652,824 for the VAE), with the small increase due to the additional input connections for the label.

3 Experimental Setup

Table 1: Training configuration.

Parameter	Value
Dataset	MNIST (60,000 train / 10,000 test)
Input dimension	784 (28×28 , flattened)
Hidden dimension	400
Latent dimension d	20 (default)
Batch size	128
Optimizer	Adam (lr = 10^{-3})
Epochs	30

All images are normalized to $[0, 1]$ by the `ToTensor()` transform, which is required for the BCE reconstruction loss. Training was performed on a single NVIDIA T4 GPU via Google Colab.

4 Results

4.1 Training Curves

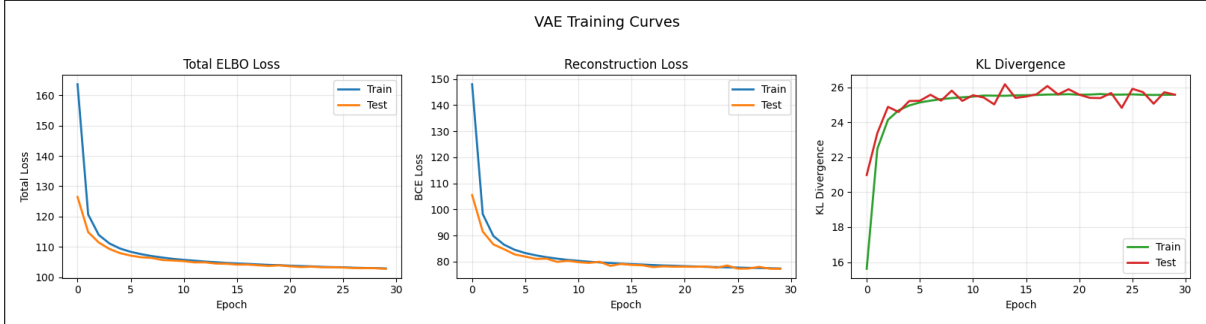


Figure 1: VAE training curves over 30 epochs. The total loss decreases from ~ 180 to ~ 103 , with reconstruction loss and KL divergence converging to approximately 79 and 24 respectively.

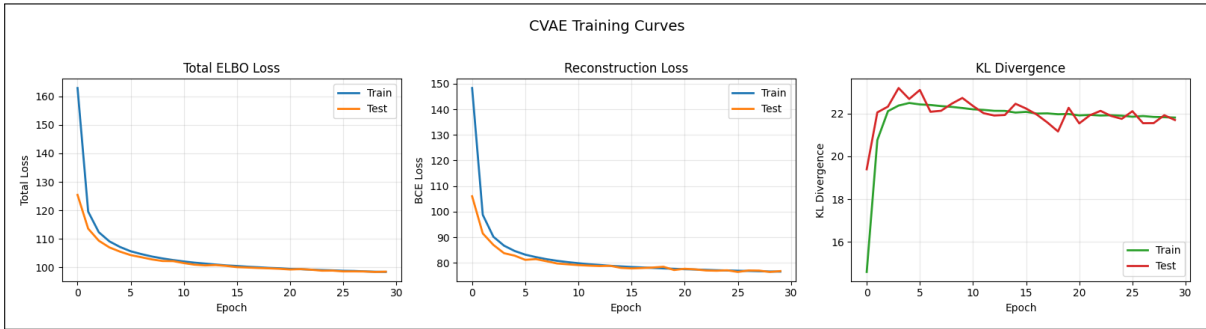


Figure 2: CVAE training curves over 30 epochs. The CVAE converges to a lower total loss (98.49) than the VAE (102.79), since the label provides additional information to the decoder.

Table 2: Final test loss comparison (after 30 epochs, $d = 20$).

Model	Total Loss	Recon (BCE)	KL Div
VAE	102.79	~ 79	~ 24
CVAE	98.49	~ 74	~ 24

The CVAE achieves a total test loss approximately 4.3 points lower than the VAE. This improvement comes almost entirely from the reconstruction term: by knowing the digit class, the decoder can focus its capacity on modeling intra-class variation (style, slant, thickness) rather than also needing to determine which digit to produce.

4.2 Reconstruction Quality

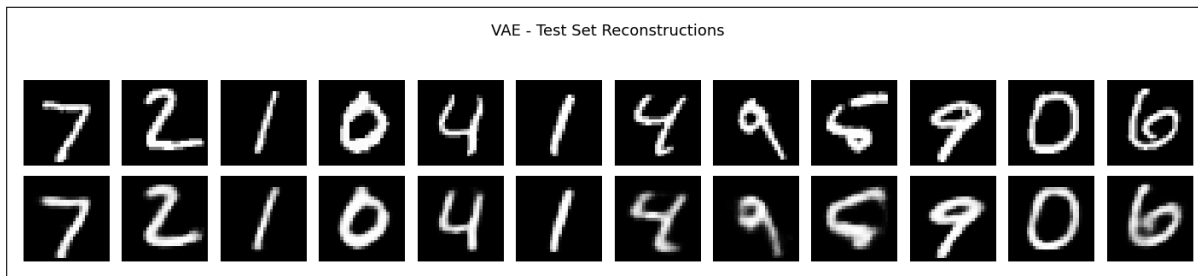


Figure 3: VAE reconstructions of test images. The model captures the overall digit shape and pose, though fine details (sharp edges, thin strokes) are smoothed out, which is a well-known artifact of the BCE loss and Gaussian decoder.

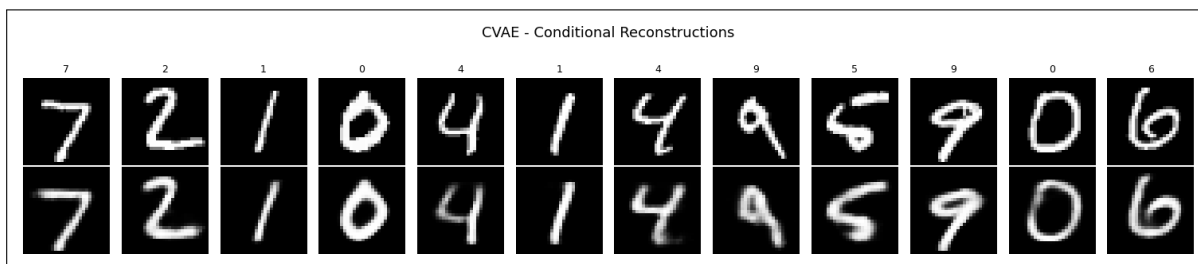


Figure 4: CVAE conditional reconstructions. Quality is comparable to the VAE, with slightly sharper reconstructions due to the label conditioning reducing the decoder’s uncertainty.

4.3 Generation

4.3.1 VAE: Unconditional Generation

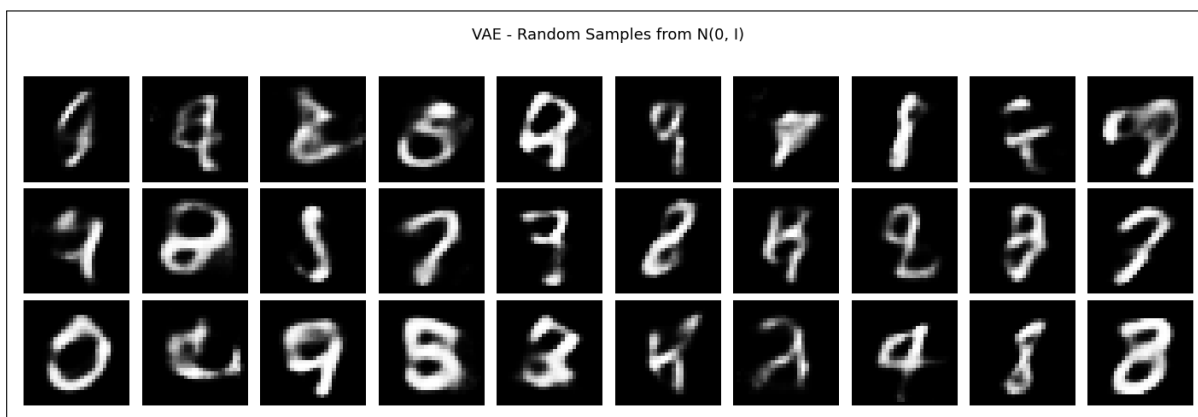


Figure 5: VAE samples generated from $z \sim \mathcal{N}(0, I)$. The digit class is uncontrolled, producing a random mix of digits. Most samples are recognizable, though some are ambiguous (e.g., blends between similar digits like 4/9 or 3/8).

4.3.2 CVAE: Conditional Generation

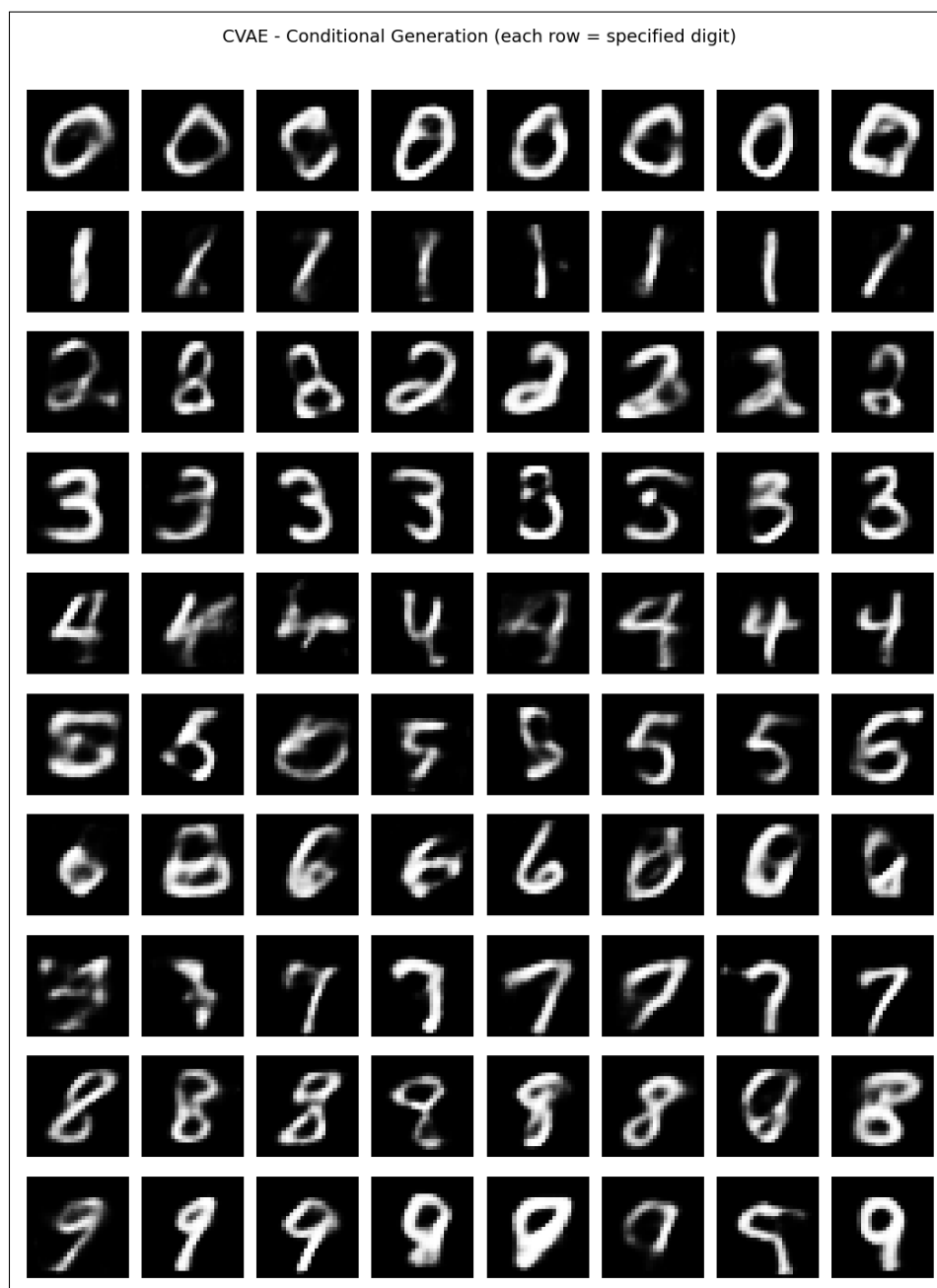


Figure 6: CVAE conditional generation. Each row corresponds to a specified digit label (0 through 9), with 8 different random latent vectors per row. The model reliably generates the requested digit class while varying the style across columns.

This is the key advantage of the CVAE: by providing the one-hot label to the decoder alongside a random z , we obtain full control over *which* digit is generated while the latent vector controls *how* it is written (style, slant, thickness).

4.3.3 Style Variation

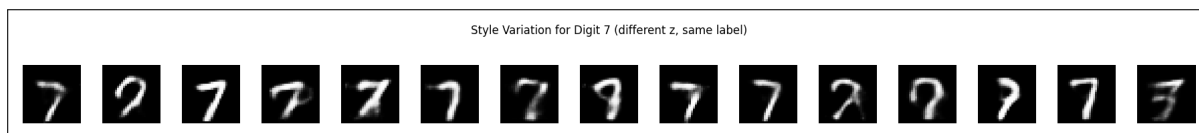


Figure 7: Style variation for digits 0, 3, and 7. All images in each row share the same label; only the latent vector z differs. The variation in thickness, slant, and proportions demonstrates that z captures meaningful intra-class variation.

4.4 VAE vs CVAE: Side-by-Side Comparison

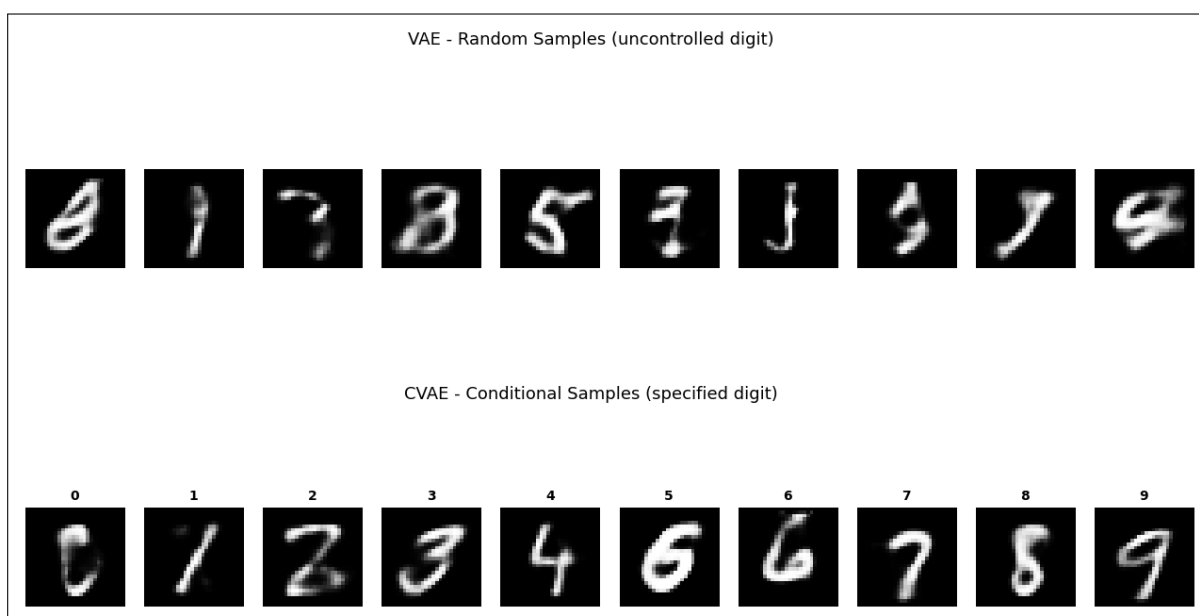


Figure 8: Side-by-side comparison. Top: VAE generates random, uncontrolled digits. Bottom: CVAE generates one of each digit (0–9) on demand. The CVAE samples are equally diverse in style but deterministic in class identity.

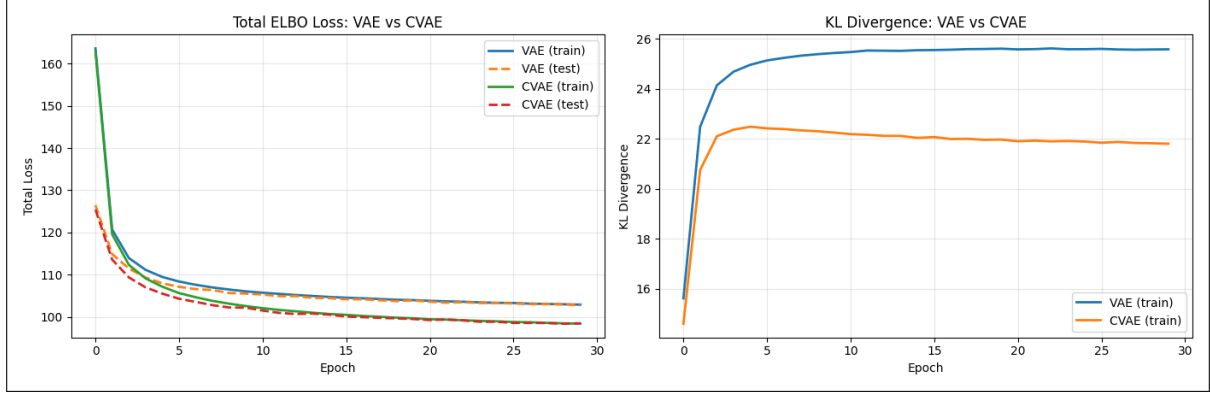


Figure 9: Loss comparison between VAE and CVAE over training. The CVAE converges to a lower total loss while maintaining similar KL divergence, indicating the improvement comes from better reconstruction.

5 Latent Space Analysis

5.1 Latent Space Traversal

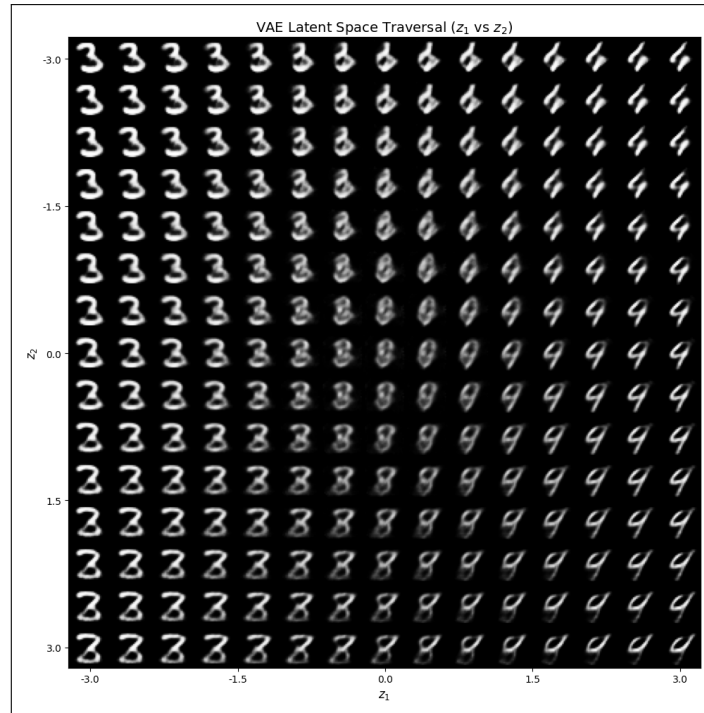


Figure 10: VAE latent space traversal. We sweep the first two latent dimensions (z_1, z_2) over $[-3, 3]$ while holding all other dimensions at 0. The smooth transitions between digit morphologies confirm that the VAE has learned a continuous, well-organized manifold.

5.2 Latent Interpolation

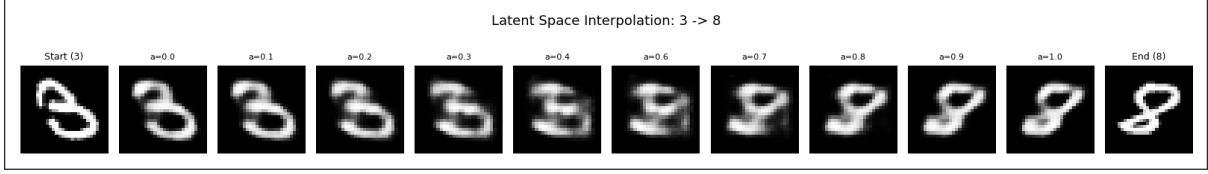


Figure 11: Linear interpolation in latent space between an encoded “3” and an encoded “8”. The transition is smooth and passes through plausible intermediate forms, confirming that the latent space does not contain sharp discontinuities between digit classes.

5.3 t-SNE Visualization

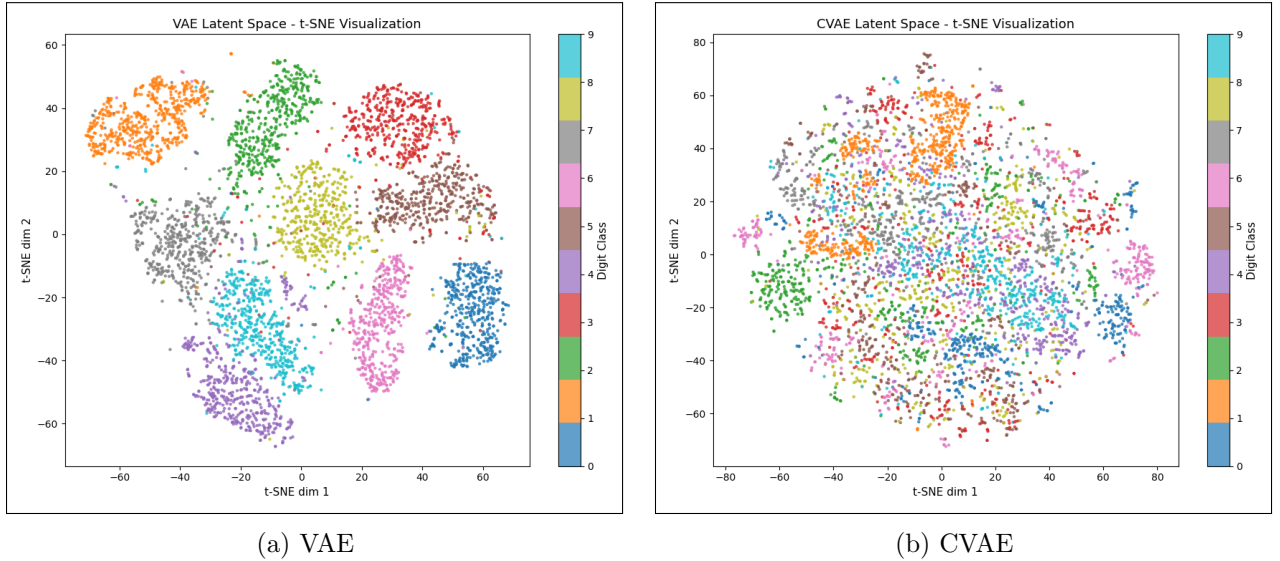


Figure 12: t-SNE projections of the 20-dimensional latent means μ for 5,000 test images, colored by digit class. Both models show clustering by digit identity. The CVAE clusters are slightly tighter because the label conditioning encourages the encoder to focus the latent space on style variation rather than class discrimination.

6 Hyperparameter Ablation: Latent Dimension

We train VAEs with latent dimensions $d \in \{2, 5, 10, 20, 50\}$ for 20 epochs each, holding all other hyperparameters fixed.

Table 3: Effect of latent dimension d on VAE test loss (20 epochs).

d	Test Loss	Recon (BCE)	KL Div
2	152.04	145.94	6.10
5	123.23	111.46	11.77
10	107.90	89.51	18.39
20	103.94	78.47	25.47
50	103.98	77.34	26.64

Key observations:

The reconstruction loss decreases monotonically with d : from 145.94 at $d=2$ to 77.34 at $d=50$. More latent dimensions give the model greater capacity to encode fine-grained details. However, the KL divergence increases correspondingly (6.10 to 26.64), since the model uses more of the available latent dimensions (pushing their posteriors away from the prior).

The total loss plateaus between $d=20$ (103.94) and $d=50$ (103.98). The extra 30 dimensions provide essentially zero benefit: the marginal improvement in reconstruction (78.47 to 77.34, a reduction of just 1.13) is almost exactly offset by the additional KL cost (25.47 to 26.64, an increase of 1.17). This indicates that $d=20$ is sufficient for MNIST, and the model naturally avoids using unnecessary dimensions.

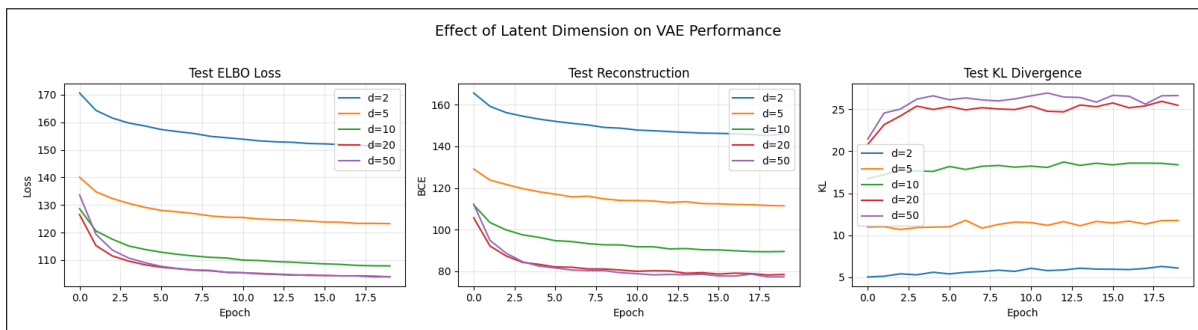


Figure 13: Effect of latent dimension on VAE test metrics. Larger d improves reconstruction at the cost of higher KL, with diminishing returns beyond $d=20$.



Figure 14: Generated samples at different latent dimensions. At $d=2$ the images are blurry and lack variety. Quality improves markedly from $d=5$ to $d=10$, with $d=20$ and $d=50$ producing comparably sharp results.

6.1 Special Case: $d = 2$ (Direct Visualization)

When the latent space is two-dimensional, we can visualize the encoder outputs directly without dimensionality reduction.

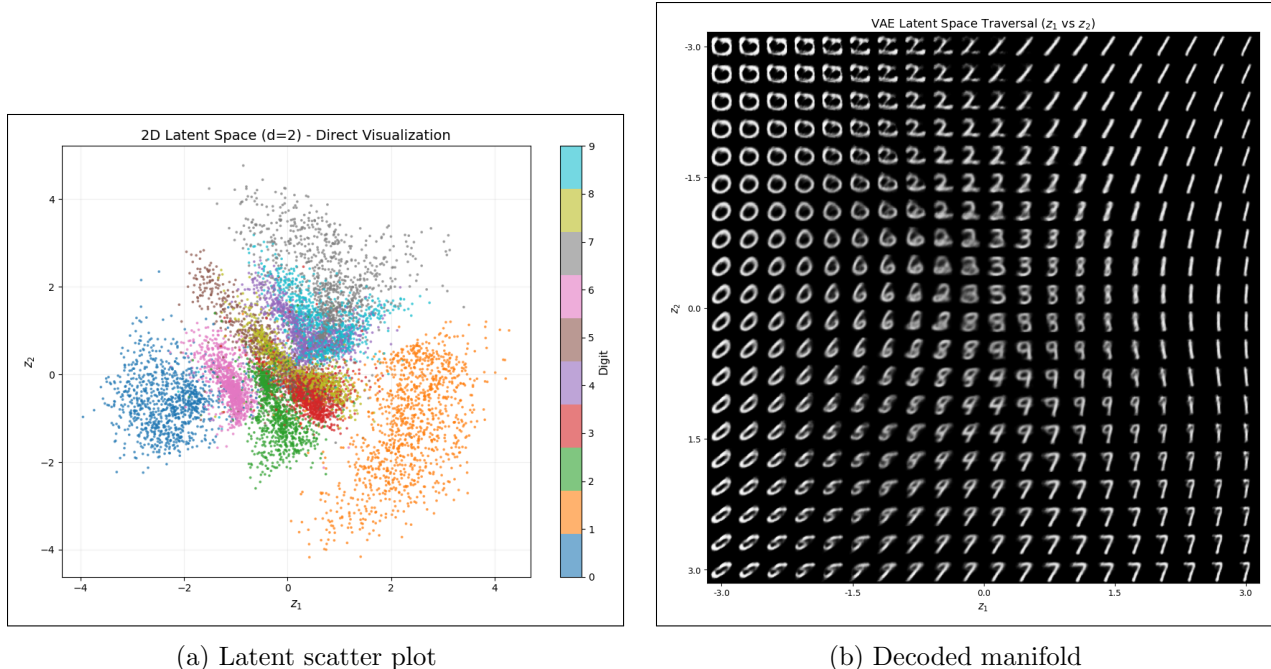


Figure 15: Direct visualization of the 2D latent space ($d=2$). Left: encoded test images show partial clustering by digit, though with significant overlap due to the bottleneck. Right: the decoded manifold reveals smooth transitions, with neighboring regions in latent space producing similar digits.

Despite the severe bottleneck, the $d=2$ model still learns a structured latent space where similar digits occupy nearby regions. However, the overlap between classes (e.g., 4/9, 3/5, 7/1) leads to the high reconstruction error seen in Table 3.

7 Discussion

Reconstruction-regularization tradeoff. The ablation results (Table 3) clearly illustrate the fundamental tension in VAE training. The reconstruction term wants the encoder to transmit as much information as possible through z , while the KL term penalizes deviation from the uninformative prior. At $d=2$, the bottleneck is so tight that reconstruction dominates the loss (145.94 out of 152.04), and the KL contribution is small (6.10). At $d=50$, the model has enough capacity that the KL term becomes the binding constraint (26.64 out of 103.98). The optimal dimension is the point where additional capacity yields no net benefit, which for MNIST is approximately $d=20$.

Blurriness of VAE samples. All generated images exhibit a characteristic softness compared to real MNIST digits. This is an inherent limitation of the combination of Gaussian decoder and pixel-wise BCE loss, which effectively averages over all possible outputs. More expressive decoders (e.g., autoregressive or diffusion-based) would produce sharper samples, but at the cost of additional complexity.

Value of conditioning. The 4.3-point improvement from VAE (102.79) to CVAE (98.49) comes from a specific source: the decoder no longer needs to infer the digit class from z alone. This frees latent capacity for style information, resulting in both better reconstruction and more controllable generation. The KL divergence remains similar between the two models, confirming that the improvement is in the decoder’s utilization of the latent code, not in the structure of the

posterior.

Latent space smoothness. The interpolation experiment (Figure 11) and the grid traversal (Figure 10) provide strong evidence that the VAE’s latent space is smooth and continuous. This is a direct consequence of the KL regularization: by encouraging $q_\phi(z|x)$ to stay close to $\mathcal{N}(0, I)$, the model avoids sharp boundaries and isolated regions in the latent space, ensuring that any sampled z decodes to a plausible image.

8 Conclusion

We implemented and evaluated a VAE and CVAE on MNIST. The key findings are:

1. The VAE learns a smooth 20-dimensional latent space and achieves a test ELBO loss of 102.79. Generated samples are recognizable but have uncontrolled digit identity.
2. The CVAE, by conditioning on one-hot class labels, achieves a lower test loss of 98.49 and enables controlled generation of specified digits with diverse styles.
3. The latent dimension ablation reveals a plateau at $d=20$: total loss is 103.94 vs 103.98 at $d=50$, with the marginal reconstruction gain exactly offset by additional KL cost.
4. Latent space visualizations (t-SNE, interpolation, grid traversal) confirm that the learned representations are continuous, semantically organized, and free of sharp discontinuities.

References

- [1] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” *Proc. ICLR*, 2014.
- [2] K. Sohn, H. Lee, and X. Yan, “Learning Structured Output Representation using Deep Conditional Generative Models,” *Proc. NeurIPS*, 2015.
- [3] C. Doersch, “Tutorial on Variational Autoencoders,” *arXiv:1606.05908*, 2016.
- [4] KL Divergence between two Gaussian distributions, <https://mr-easy.github.io/2020-04-16-kl-divergence-between-2-gaussian-distributions/>